UNIT-2

Descriptive statistics, data preparation. Exploratory Data Analysis data summarization, data distribution, measuring asymmetry. Sample and estimated mean, variance and standard score. Statistical Inference frequency approach, variability of estimates, hypothesis testing using confidence intervals, using p-values

DescriptiveStatistics

Descriptive statistics helps to simplify large amounts of data in a sensible way. In contrast to inferential statistics, which will be introduced in a later chapter, in descriptive statistics we do not draw conclusions beyond the data we are analyzing; neither do we reach any conclusions regarding hypotheses we may make. We do nottry to infer characteristics of the "population" (see below) of the data, but claim to present quantitative descriptions of it in a manageable form. It is simply a way to describe the data.

Statistics, and in particular descriptive statistics, is based on two main concepts:

- a *population* is a collection of objects, items ("units") about which information issought;
- a *sample* is a part of the population that is observed.

Descriptive statistics applies the concepts, measures, and terms that are used to describe the basic features of the samples in a study. These procedures are essential to provide summaries about the samples as an approximation of the population. Together with simple graphics, they form the basis of every quantitative analysis ofdata. In order to describe the sample data and to be able to infer any conclusion, weshould go through several steps:

1. *Data preparation*: Given a specific example, we need to prepare the data forgenerating statistically valid descriptions.
2. *Descriptive statistics*: This generates different statistics to describe and summa-rize the data concisely and evaluate different ways to visualize them.

Data Preparation

One of the first tasks when analyzing data is to collect and prepare the data in a format appropriate for analysis of the samples. The most common steps for data preparationinvolve the following operations.

1. *Obtaining the data:* Data can be read directly from a file or they might be obtained by scraping the web.
2. *Parsing the data:* The right parsing procedure depends on what format the dataare in: plain text, fixed columns, CSV, XML, HTML, etc.
3. *Cleaning the data:* Survey responses and other data files are almost always incomplete. Sometimes, there are multiple codes for things such as, not asked, didnot know, and declined to answer. And there are almost always errors. A simplestrategy is to remove or ignore incomplete records.
4. *Building data structures:* Once you read the data, it is necessary to store them ina data structure that lends itself to the analysis we are interested in. If the data fitinto the memory, building a data structure is usually the way to go. If not, usually a database is built, which is an out-of-memory data structure. Most databases provide a mapping from keys to values, so they serve as dictionaries.

The Adult Example

Let us consider a public database called the "Adult" dataset, hosted on the UCI's Machine Learning Repository.[1] It contains approximately 32,000 observations concerning different financial parameters related to the US population: age, sex, marital(marital status of the individual), country, income (Boolean variable: whether the per- son makes more than $50,000 per annum), education (the highest level of educationachieved by the individual), occupation, capital gain, etc.

We will show that we can explore the data by asking questions like: "Are men more likely to become high-income professionals than women, i.e., to receive an income of over $50,000 per annum?"

First, let us read the data:

In[1]:

```
file = open('files/ch03/adult.data', 'r')
def chr_int(a):
    if a.isdigit(): return int(a)
    else: return 0

data = []
for line in file:
     data1 = line.split(', ')
     if len(data1) == 15:
         data.append([chr_int(data1[0]), data1[1],
                      chr_int(data1[2]), data1[3],
                      chr_int(data1[4]), data1[5],
                      data1[6], data1[7], data1[8],
                      data1[9], chr_int(data1[10]),
                      chr_int(data1[11]),
                      chr_int(data1[12]),
                      data1[13], data1[14]
                     ])
```

Checking the data, we obtain:

In[2]:

```
print data[1:2]
```

Out[2]: [[50, 'Self-emp-not-inc', 83311, 'Bachelors', 13,
'Married-civ-spouse', 'Exec-managerial', 'Husband', 'White','Male', 0, 0, 13, 'United-States', ' <= 50K']]

One of the easiest ways to manage data in Python is by using the DataFrame structure, defined in the *Pandas* library, which is a two-dimensional, size-mutable,potentially heterogeneous tabular data structure with labeled axes:

In[3]:

```
df = pd.DataFrame(data)
df.columns = [
    'age', 'type_employer', 'fnlwgt',
    'education', 'education_num', 'marital',
    'occupation',' relationship', 'race',
    'sex', 'capital_gain', 'capital_loss',
    'hr_per_week', 'country', 'income'
    ]
```

The command shapegives exactly the number of data samples (in rows, in this case) and features (in columns):

In[4]:

```
df.shape
```

Out[4]: (32561, 15)

Thus, we can see that our dataset contains 32,561 data records with 15 featureseach. Let us count the number of items per country:

In[5]:
```
counts = df.groupby('country').size()
print counts.head()
```

Out[5]: country
? 583
Cambodia 19
Vietnam 67
Yugoslavia 16

The first row shows the number of samples with unknown country, followed bythe number of samples corresponding to the first countries in the dataset.

Let us split people according to their gender into two groups: men and women.

In[6]:
```
ml = df[(df.sex == 'Male')]
```

If we focus on high-income professionals separated by sex, we can do:

In[7]:
```
ml1 = df[(df.sex == 'Male') & (df.income =='>50K\n')
    ]
fm = df[(df.sex == 'Female')]
fml = df[(df.sex == 'Female') & (df.income =='>50K\n
    ')]
```

Exploratory Data Analysis

The data that come from performing a particular measurement on all the subjects in a sample represent our observations for a single characteristic like country, age, education, etc. These measurements and categories represent a *sample distribution* of the variable, which in turn approximately represents the *population distribution* of the variable. One of the main goals of exploratory data analysis is to visualize and summarize the sample distribution, thereby allowing us to make tentative assumptions about the population distribution.

Summarizing the Data

The data in general can be categorical or quantitative. For categorical data, a simple tabulation of the frequency of each category is the best non-graphical exploration for data analysis. For example, we can ask ourselves what is the proportion of high-income professionals in our database:

In[8]:

```
df1 = df[(df.income=='>50K\n')]
print 'The rate of people with high income is: ',
      int(len(df1)/float(len(df))*100), '%.'
print 'The rate of men with high income is: ',
      int(len(ml1)/float(len(ml))*100), '%.'
print 'The rate of women with high income is: ',
      int(len(fm1)/float(len(fm))*100), '%.'
```

Out[8]: The rate of people with high income is: 24 %.
The rate of men with high income is: 30 %. The rate of women with high income is: 10 %.

Given a quantitative variable, exploratory data analysis is a way to make prelim-inary assessments about the population distribution of the variable using the data of the observed samples. The characteristics of the population distribution of a quanti- tative variable are its *mean*, *deviation*, *histograms*, *outliers*, etc. Our observed data represent just a finite set of samples of an often infinite number of possible samples. The characteristics of our randomly observed samples are interesting only to the degree that they represent the population of the data they came from.

Mean

One of the first measurements we use to have a look at the data is to obtain *samplestatistics* from the data, such as the sample mean [1]. Given a sample of n values, $\{x_i, i = 1, \ldots, n\}$, the *mean*, μ, is the sum of the values divided by the number of values,[2] in other words:

$$\mu = \frac{1}{n} \sum_{i=1}^{n} x_i . \tag{3.1}$$

The terms mean and *average* are often used interchangeably. In fact, the maindistinction between them is that the mean of a sample is the summary statistic com-puted by Eq. (3.1), while an average is not strictly defined and could be one of manysummary statistics that can be chosen to describe the central tendency of a sample.

In our case, we can consider what the average age of men and women samples inour dataset would be in terms of their mean:

Descriptive Statistics

In[9]:

```
print 'The average age of men is: ',
      ml['age'].mean()
print 'The average age of women is: ',
      fm['age'].mean()

print 'The average age of high-income men is: ',
      ml1['age'].mean()
print 'The average age of high-income women is: ',
      fm1['age'].mean()
```

Out[9]:

```
The average age of men is: 39.4335474989 The average age of
women is: 36.8582304336
The average age of high-income men is: 44.6257880516
The average age of high-income women is: 42.1255301103
```

This difference in the sample means can be considered initial evidence that thereare differences between men and women with high income!

Comment: Later, we will work with both concepts: the population mean and thesample mean. We should not confuse them! The first is the mean of samples takenfrom the population; the second, the mean of the whole population.

Sample Variance

The mean is not usually a sufficient descriptor of the data. We can go further by knowing two numbers: mean and *variance*. The variance σ^2 describes the spread ofthe data and it is defined as follows:

$$\sigma^2 = \frac{1}{n} \sum_{i} (x_i - \mu)^2. \tag{3.2}$$

The term $(x_i - \mu)$ is called the *deviation* from the mean, so the variance is the mean squared deviation. The square root of the variance, σ, is called the *standard deviation*. We consider the standard deviation, because the variance is hard to interpret (e.g., ifthe units are grams, the variance is in grams squared).

Let us compute the mean and the variance of hours per week men and women inour dataset work:

In[10]:

```
ml_mu = ml['age'].mean()
fm_mu = fm['age'].mean()
ml_var = ml['age'].var()
fm_var = fm['age'].var()
ml_std = ml['age'].std()
fm_std = fm['age'].std()
print 'Statistics of age for men: mu:',
```

Out[10]: Statistics of age for men: mu: 39.4335474989 var: 178.773751745std: 13.3706301925
Statistics of age for women: mu: 36.8582304336 var:196.383706395 std: 14.0136970994

We can see that the mean number of hours worked per week by women is significantly lesser than that worked by men, but with much higher variance and standarddeviation.

Sample Median

The mean of the samples is a good descriptor, but it has an important drawback: what will happen if in the sample set there is an error with a value very different from the rest? For example, considering hours worked per week, it would normally be in a range between 20 and 80; but what would happen if by mistake there was a value of 1000? An item of data that is significantly different from the rest of the data is called an *outlier*. In this case, the mean, μ, will be drastically changed towards the outlier. One solution to this drawback is offered by the statistical *median*, μ_{12}, which is an order statistic giving the middle value of a sample. In this case, all the values are ordered by their magnitude and the median is defined as the value that is in themiddle of the ordered list. Hence, it is a value that is much more robust in the face of outliers.

Let us see, the median age of working men and women in our dataset and the median age of high-income men and women:

In[11]:
```
ml_median = ml['age'].median()
fm_median = fm['age'].median()
print "Median age per men and women: ",
      ml_median, fm_median

ml_median_age = ml1['age'].median()
fm_median_age = fm1['age'].median()
print "Median age per men and women with high-
    income: ",
      ml_median_age, fm_median_age
```

Out[11]: Median age per men and women: 38.0 35.0
Median age per men and women with high-income: 44.0 41.0

As expected, the median age of high-income people is higher than the whole setof working people, although the difference between men and women in both sets isthe same.

Quantiles and Percentiles

Sometimes we are interested in observing how sample data are distributed in general. In this case, we can order the samples x_i , then find the x_p so that it divides the datainto two parts, where:

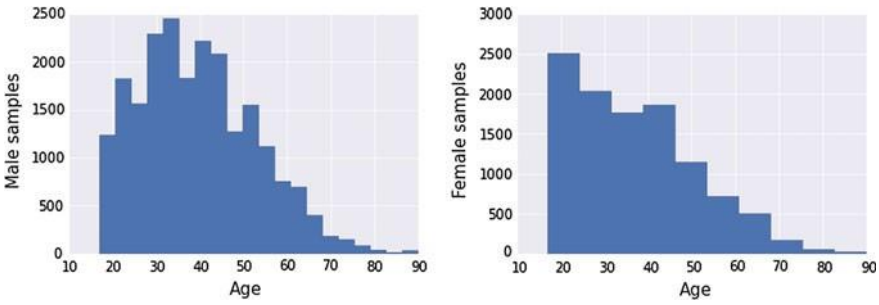


Fig. 3.1 Histogram of the age of working men (*left*) and women (*right*)

- a fraction p of the data values is less than or equal to x_p and
- the remaining fraction $(1 - p)$ is greater than x_p.

That value, x_p, is the *p-th* quantile, or the 100 p-*th* percentile. For example, a 5-number summary is defined by the values x_{min}, Q_1, Q_2, Q_3, x_{max}, where Q_1 is the 25 *p-th* percentile, Q_2 is the 50 *p-th* percentile and Q_3 is the 75 *p-th* percentile.

Data Distributions

Summarizing data by just looking at their mean, median, and variance can be dangerous: very different data can be described by the same statistics. The best thing to do is to validate the data by inspecting them. We can have a look at the data distribution, which describes how often each value appears (i.e., what is its frequency).

The most common representation of a distribution is a *histogram*, which is a graph that shows the frequency of each value. Let us show the age of working men and women separately.

In[12]:
```
ml_age = ml['age']
ml_age.hist(normed = 0, histtype = 'stepfilled',
    bins = 20)
```

In[13]:
```
fm_age = fm['age']
fm_age.hist(normed = 0, histtype = 'stepfilled',
    bins = 10)
```

The output can be seen in Fig. 3.1. If we want to compare the histograms, we canplot them overlapping in the same graphic as follows:

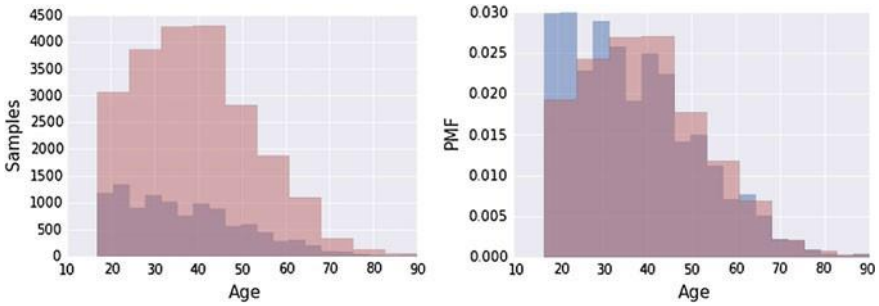


Fig. 3.2 Histogram of the age of working men (in *ochre*) and women (in *violet*) (*left*). Histogram of the age of working men (in *ochre*), women (in *blue*), and their intersection (in *violet*) after samples normalization (*right*)

In[14]:

```
import seaborn as sns
fm_age.hist(normed = 0, histtype = 'stepfilled',
            alpha = .5, bins = 20)
ml_age.hist(normed = 0, histtype = 'stepfilled',
            alpha = .5,
            color = sns.desaturate("indianred",
                .75),
            bins = 10)
```

The output can be seen in Fig. 3.2 (left). Note that we are visualizing the absolute values of the number of people in our dataset according to their age (the abscissa ofthe histogram). As a side effect, we can see that there are many more men in these conditions than women.

We can normalize the frequencies of the histogram by dividing/normalizing by n, the number of samples. The normalized histogram is called the *Probability MassFunction* (PMF).

In[15]:

```
fm_age.hist(normed = 1, histtype = 'stepfilled',
            alpha = .5, bins = 20)
ml_age.hist(normed = 1, histtype = 'stepfilled',
            alpha = .5, bins = 10,
            color = sns.desaturate("indianred",
                .75))
```

This outputs Fig. 3.2 (right), where we can observe a comparable range of individuals (men and women).

The *Cumulative Distribution Function* (CDF), or just distribution function, describes the probability that a real-valued random variable X with a given probability distribution will be found to have a value less than or equal to x. Let us show the CDF of age distribution for both men and women.

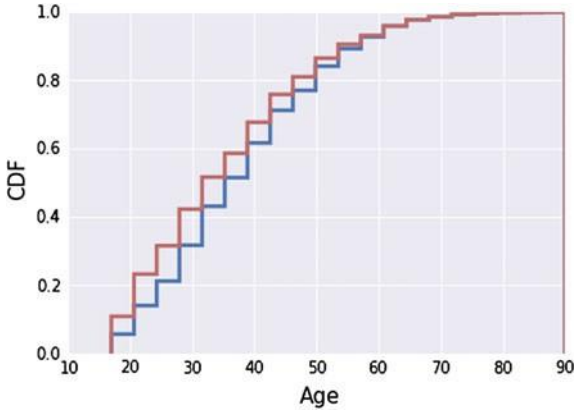


Fig. 3.3 The CDF of the ageof working male (in *blue*) and female (in *red*) samples

In[16]:

```
ml_age.hist(normed = 1, histtype = 'step',
            cumulative = True, linewidth = 3.5,
            bins = 20)
fm_age.hist(normed = 1, histtype='step',
            cumulative = True, linewidth = 3.5,
            bins = 20,
            color = sns.desaturate("indianred",
                .75))
```

The output can be seen in Fig. 3.3, which illustrates the CDF of the age distributions for both men and women.

Outlier Treatment

As mentioned before, outliers are data samples with a value that is far from the centraltendency. Different rules can be defined to detect outliers, as follows:

- Computing samples that are far from the median.
- Computing samples whose values exceed the mean by 2 or 3 standard deviations.

For example, in our case, we are interested in the age statistics of men versus women with high incomes and we can see that in our dataset, the minimum age is 17years and the maximum is 90 years. We can consider that some of these samples are due to errors or are not representable. Applying the domain knowledge, we focus onthe median age (37, in our case) up to 72 and down to 22 years old, and we considerthe rest as outliers.

In[17]:
```
df2 = df.drop(df.index[
    (df.income == '>50K\n') &
    (df['age'] > df['age'].median() + 35) &
    (df['age'] > df['age'].median() -15)
    ])
ml1_age = ml1['age']
fm1_age = fm1['age']

ml2_age = ml1_age.drop(ml1_age.index[
    (ml1_age > df['age'].median() + 35) &
    (ml1_age > df['age'].median() - 15)
    ])
fm2_age = fm1_age.drop(fm1_age.index[
    (fm1_age > df['age'].median() + 35) &
    (fm1_age > df['age'].median() - 15)
    ])
```

We can check how the mean and the median changed once the data were cleaned:

In[18]:
```
mu2ml = ml2_age.mean()
std2ml = ml2_age.std()
md2ml = ml2_age.median()
mu2fm = fm2_age.mean()
std2fm = fm2_age.std()
md2fm = fm2_age.median()

print "Men statistics:"
print "Mean:", mu2ml, "Std:", std2ml
print "Median:", md2ml
print "Min:", ml2_age.min(), "Max:", ml2_age.max()

print "Women statistics:"
print "Mean:", mu2fm, "Std:", std2fm
print "Median:", md2fm
print "Min:", fm2_age.min(), "Max:", fm2_age.max()
```

Out[18]: Men statistics: Mean: 44.3179821239 Std: 10.0197498572 Median: 44.0 Min: 19 Max: 72
Women statistics: Mean: 41.877028181 Std: 10.0364418073 Median: 41.0 Min: 19 Max: 72

Let us visualize how many outliers are removed from the whole data by:

In[19]:
```
plt.figure(figsize = (13.4, 5))
df.age[(df.income == '>50K\n')]
      .plot(alpha = .25, color = 'blue')
df2.age[(df2.income == '>50K\n')]
      .plot(alpha = .45, color = 'red')
```

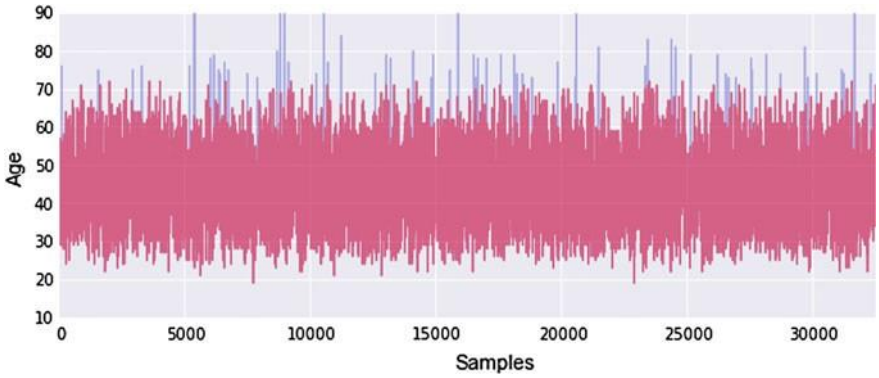


Fig. 3.4 The *red* shows the cleaned data without the considered outliers (in *blue*)

Figure 3.4 shows the outliers in blue and the rest of the data in red. Visually, wecan confirm that we removed mainly outliers from the dataset.

Next we can see that by removing the outliers, the difference between the popula-tions (men and women) actually decreased. In our case, there were more outliers inmen than women. If the difference in the mean values before removing the outliersis 2.5, after removing them it slightly decreased to 2.44:

In[20]:
```
print 'The mean difference with outliers is: %4.2f.
    '
      % (ml_age.mean() - fm_age.mean())
print 'The mean difference without outliers is:
    %4.2f.'
      % (ml2_age.mean() - fm2_age.mean())
```

Out[20]:
```
The mean difference with outliers is: 2.58.
The mean difference without outliers is: 2.44.
```

Let us observe the difference of men and women incomes in the cleaned subsetwith some more details.

In[21]:
```
countx, divisionx = np.histogram(ml2_age, normed =
    True)
county, divisiony = np.histogram(fm2_age, normed =
    True)

val = [(divisionx[i] + divisionx[i+1])/2
       for i in range(len(divisionx) - 1)]
plt.plot(val, countx - county, 'o-')
```

The results are shown in Fig. 3.5. One can see that the differences between maleand female values are slightly negative before age 42 and positive after it. Hence, women tend to be promoted (receive more than 50 K) earlier than men.

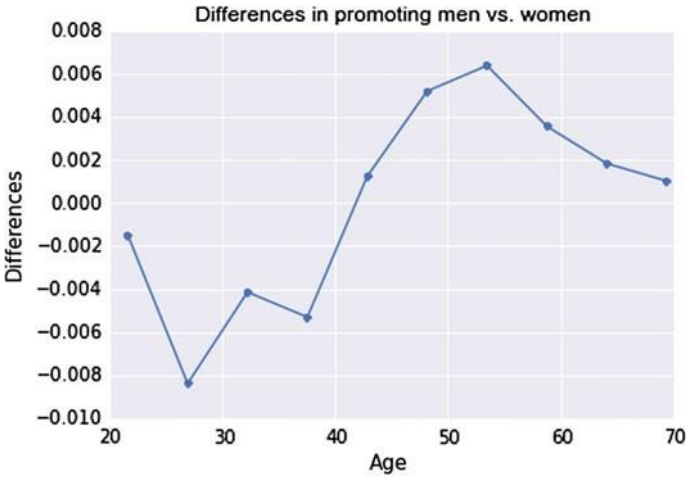


Fig. 3.5 Differences in high-income earner men versus women as a function of age

Measuring Asymmetry: Skewness and Pearson's Median Skewness Coefficient

For univariate data, the formula for *skewness* is a statistic that measures the asym-metry of the set of n data samples, x_i:

$$g_1 = \frac{1}{n}\frac{\sum_i (x_i - \mu^3)}{\sigma^3}, \tag{3.3}$$

where μ is the mean, σ is the standard deviation, and n is the number of data points.

Negative deviation indicates that the distribution "skews left" (it extends further to the left than to the right). One can easily see that the skewness for a normal distribution is zero, and any symmetric data must have a skewness of zero. Note that skewness can be affected by outliers! A simpler alternative is to look at the relationship between the mean μ and the median μ_{12}.

In[22]:

```
def skewness(x):
    res = 0
    m = x.mean()
    s = x.std()
    for i in x:
        res += (i-m) * (i-m) * (i-m)
    res /= (len(x) * s * s * s)
    return res

print "Skewness of the male population = ",
      skewness(ml2_age)
print "Skewness of the female population is = ",
      skewness(fm2_age)
```

Out[22]: Skewness of the male population = 0.266444383843 Skewness of the female population = 0.386333524913

That is, the female population is more skewed than the male, probably since mencould be most prone to retire later than women.

The **Pearson's median skewness coefficient** is a more robust alternative to the skewness coefficient and is defined as follows:

$$g_p = 3(\mu - \mu_{12})\sigma.$$

There are many other definitions for skewness that will not be discussed here. In our case, if we check the Pearson's skewness coefficient for both men and women,we can see that the difference between them actually increases:

In[23]:

```
def pearson(x):
    return 3*(x.mean() - x.median())*x.std()

print "Pearson's coefficient of the male population
    = ",
      pearson(ml2_age)
print "Pearson's coefficient of the female
   population = ",
      pearson(fm2_age)
```

Out[23]: Pearson's coefficient of the male population = 9.55830402221 Pearson's coefficient of the female population = 26.4067269073

Continuous Distribution

The distributions we have considered up to now are based on empirical observationsand thus are called *empirical distributions*. As an alternative, we may be interested in considering distributions that are defined by a continuous function and are called*continuous distributions* [2]. Remember that we defined the PMF, $f_X(x)$, of a discreterandom variable X as $f_X(x) = P(X = x)$ for all x. In the case of a continuous random variable X, we speak of the *Probability Density Function* (PDF), which

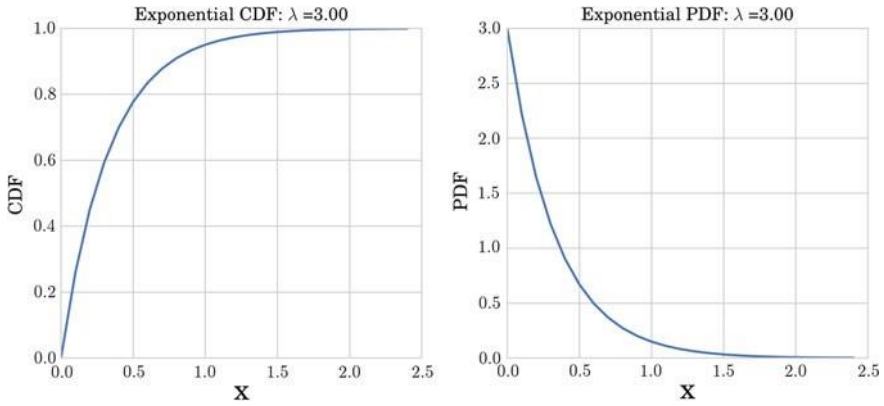


Fig. 3.6 Exponential CDF (*left*) and PDF (*right*) with $\lambda = 3.00$

is defined as $F_X(x)$ where this satisfies: $F_X(x) = \int_{-\infty}^{x} f_X(t)\delta t$ for all x. There are many continuous distributions; here, we will consider the most common ones: the exponential and the normal distributions.

The Exponential Distribution

Exponential distributions are well known since they describe the inter-arrival time between events. When the events are equally likely to occur at any time, the distri-bution of the inter-arrival time tends to an exponential distribution. The CDF and the PDF of the exponential distribution are defined by the following equations:

$$CDF(x) = 1 - e^{-\lambda x}, \qquad PDF(x) = \lambda e^{-\lambda x}.$$

The parameter λ defines the shape of the distribution. An example is given inFig. 3.6. It is easy to show that the mean of the distribution is $\frac{1}{\lambda}$, the variance is $\frac{1}{\lambda^2}$ and the median is $\frac{\ln 2}{\lambda}$.

Note that for a small number of samples, it is difficult to see that the exact empirical distribution fits a continuous distribution. The best way to observe this match is to generate samples from the continuous distribution and see if these samples match the data. As an exercise, you can consider the birthdays of a large enough group of people, sorting them and computing the inter-arrival time in days. If you plot the CDF of the inter-arrival times, you will observe the exponential distribution.

There are a lot of real-world events that can be described with this distribution, including the time until a radioactive particle decays; the time it takes before your next telephone call; and the time until default (on payment to company debt holders)in reduced-form credit risk modeling. The random variable X of the lifetime of some batteries is associated with a probability density function of the form: $PDF(x) = \frac{1}{4}e^{-\frac{x}{4}}e^{-\frac{(x-\mu)^2}{2}}$

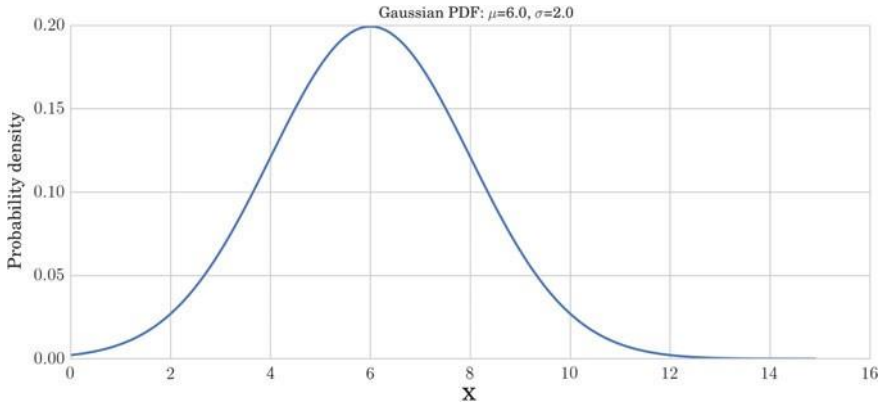


Fig. 3.7 Normal PDF with $\mu = 6$ and $\sigma = 2$

The Normal Distribution

The *normal distribution*, also called the *Gaussian distribution*, is the most common since it represents many real phenomena: economic, natural, social, and others. Some well-known examples of real phenomena with a normal distribution are as follows:

- The size of living tissue (length, height, weight).
- The length of inert appendages (hair, nails, teeth) of biological specimens.
- Different physiological measurements (e.g., blood pressure), etc.

The normal CDF has no closed-form expression and its most common represen-tation is the PDF:

$$\text{PDF}(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}.$$

The parameter σ defines the shape of the distribution. An example of the PDF ofa normal distribution with $\mu = 6$ and $\sigma = 2$ is given in Fig. 3.7.

Kernel Density

In many real problems, we may not be interested in the parameters of a particular distribution of data, but just a continuous representation of the data. In this case, we should estimate the distribution non-parametrically (i.e., making no assumptionsabout the form of the underlying distribution) using kernel density estimation. Let us imagine that we have a set of data measurements without knowing their distribution and we need to estimate the continuous representation of their distribution. In this case, we can consider a Gaussian kernel to generate the density around the data. Letus consider a set of random data generated by a bimodal normal distribution. If we consider a Gaussian kernel around the data, the sum of those kernels can give us

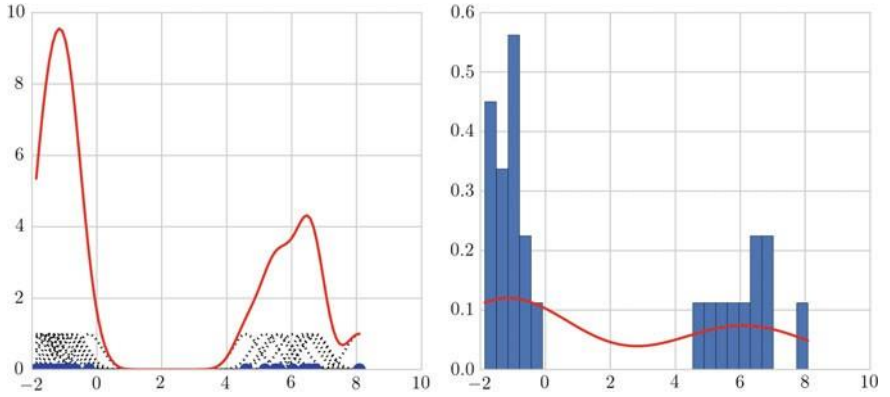


Fig. 3.8 Summed kernel functions around a random set of points (*left*) and the kernel density estimate with the optimal bandwidth (*right*) for our dataset. Random data shown in *blue*, kernel shown in *black* and summed function shown in *red*

a continuous function that when normalized would approximate the density of the distribution:

In[24]:

```
x1 = np.random.normal(-1, 0.5, 15)
x2 = np.random.normal(6, 1, 10)
y = np.r_[x1, x2] # r_ translates slice objects to
    concatenation along the first axis.
x = np.linspace(min(y), max(y), 100)

s = 0.4 # Smoothing parameter

# Calculate the kernels
kernels = np.transpose([norm.pdf(x, yi, s) for yi
    in y])
plt.plot(x, kernels, 'k:')
plt.plot(x, kernels.sum(1), 'r')
plt.plot(y, np.zeros(len(y)), 'bo', ms = 10)
```

Figure 3.8 (left) shows the result of the construction of the continuous functionfrom the kernel summarization.

In fact, the library SciPy[3] implements a Gaussian kernel density estimation that automatically chooses the appropriate bandwidth parameter for the kernel. Thus, thefinal construction of the density estimate will be obtained by:

.

In[25]:

```
from scipy.stats import kde
density = kde.gaussian_kde(y)
xgrid = np.linspace(x.min(), x.max(), 200)
plt.hist(y, bins = 28, normed = True)
plt.plot(xgrid, density(xgrid), 'r-')
```

Figure 3.8 (right) shows the result of the kernel density estimate for our example.

Estimation

An important aspect when working with statistical data is being able to use estimatesto approximate the values of unknown parameters of the dataset. In this section, we will review different kinds of estimators (estimated mean, variance, standard score,etc.).

Sample and Estimated Mean, Variance and Standard Scores

In continuation, we will deal with point estimators that are single numerical estimates of parameters of a population.

Mean

Let us assume that we know that our data are coming from a normal distribution andthe random samples drawn are as follows:

$$\{0.33, -1.76, 2.34, 0.56, 0.89\}.$$

The question is can we guess the mean μ of the distribution? One approximation isgiven by the sample mean, $\bar{x}$. This process is called *estimation* and the statistic (e.g., the sample mean) is called an *estimator*. In our case, the sample mean is 0.472, and it seems a logical choice to represent the mean of the distribution. It is not so evident ifwe add a sample with a value of 465. In this case, the sample mean will be 77.11, which does not look like the mean of the distribution. The reason is due to the fact that the last value seems to be an outlier compared to the rest of the sample. In orderto avoid this effect, we can try first to remove outliers and then to estimate the mean; or we can use the sample median as an estimator of the mean of the distribution. If there are no outliers, the sample mean x minimizes the following *mean squared error*:

$$MSE = \frac{1}{n} \sum (x - \mu)^2,$$

where n is the number of times we estimate the mean.Let us compute the MSE of a set of random data:

In[26]:
```
NTs = 200
mu = 0.0
var = 1.0
err = 0.0
NPs = 1000
for i in range(NTs):
    x = np.random.normal(mu, var, NPs)
    err += (x.mean()-mu)**2
print 'MSE: ', err/NTests
```

Out[26]: MSE: 0.00019879541147

Variance

If we ask ourselves what is the variance, σ^2, of the distribution of X, analogously we can use the sample variance as an estimator. Let us denote by σ^2 the sample varianceestimator:

$$\sigma^2 = \frac{1}{n} \sum_i (x_i - \bar{x})^2.$$

For large samples, this estimator works well, but for a small number of samplesit is biased. In those cases, a better estimator is given by:

$$\tilde{\sigma}^2 = \frac{1}{n-1} \sum_i (x_i - \bar{x})^2.$$

Standard Score

In many real problems, when we want to compare data, or estimate their correlations or some other kind of relations, we must avoid data that come in different units. For example, weight can come in kilograms or grams. Even data that come in the same units can still belong to different distributions. We need to normalize them tostandard scores. Given a dataset as a series of values, $\{x_i\}$, we convert the data to standard scores by subtracting the mean and dividing them by the standard deviation:

$$z_i = \frac{(x_i - \mu)}{\sigma}.$$

Note that this measure is dimensionless and its distribution has a mean of 0 and variance of 1. It inherits the "shape" of the dataset: if X is normally distributed, so is Z; if X is skewed, so is Z.

Covariance, and Pearson's and Spearman's Rank Correlation

Variables of data can express relations. For example, countries that tend to invest

in research also tend to invest more in education and health. This kind of relationshipis captured by the covariance.

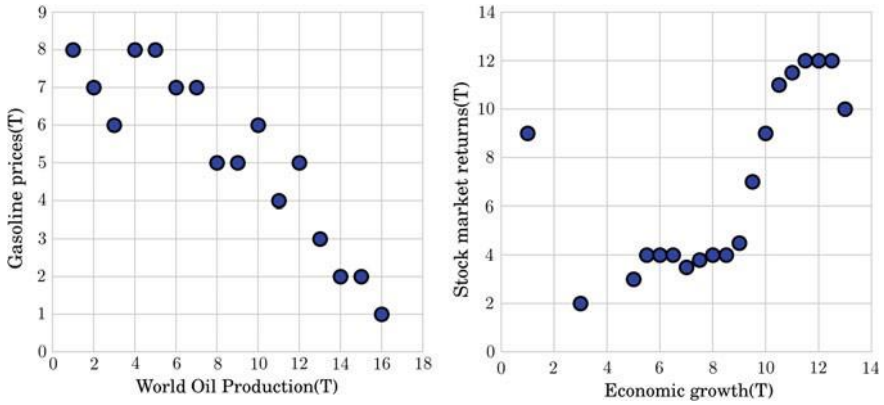


Fig. 3.9 Positive correlation between economic growth and stock market returns worldwide (*left*). Negative correlation between the world oil production and gasoline prices worldwide (*right*)

Covariance

When two variables share the same tendency, we speak about *covariance*. Let us consider two series, $\{x_i\}$ and $\{y_i\}$. Let us center the data with respect to their mean: $dx_i = x_i - \mu_X$ and $dy_i = y_i - \mu_Y$. It is easy to show that when $\{x_i\}$ and $\{y_i\}$ vary together, their deviations tend to have the same sign. The covariance is defined as the mean of the following products:

$$Cov(X,Y) = \frac{1}{n}\sum_{i=1}^{n} dx_i\, dy_i ,$$

where n is the length of both sets. Still, the covariance itself is hard to interpret.

Correlation and the Pearson's Correlation

If we normalize the data with respect to their deviation, that leads to the standardscores; and then multiplying them, we get:

$$\rho_i = \frac{x_i - \mu_X}{\sigma_X}\,\frac{y_i - \mu_Y}{\sigma_Y} .$$

The mean of this product is $\rho = \frac{1}{n}\sum_{i=1}^{n} \rho_i$. Equivalently, we can rewrite ρ in terms of the covariance, and thus obtain the *Pearson's correlation*:

$$\rho = \frac{Cov(X,Y)}{\sigma_X \sigma_Y} .$$

Note that the Pearson's correlation is always between -1 and $+1$, where themagnitude depends on the degree of correlation. If the Pearson's correlation is -1 (or1), it means that the variables are perfectly correlated (positively or negatively) (see Fig. 3.9). This means that one variable can predict the other very well. However,

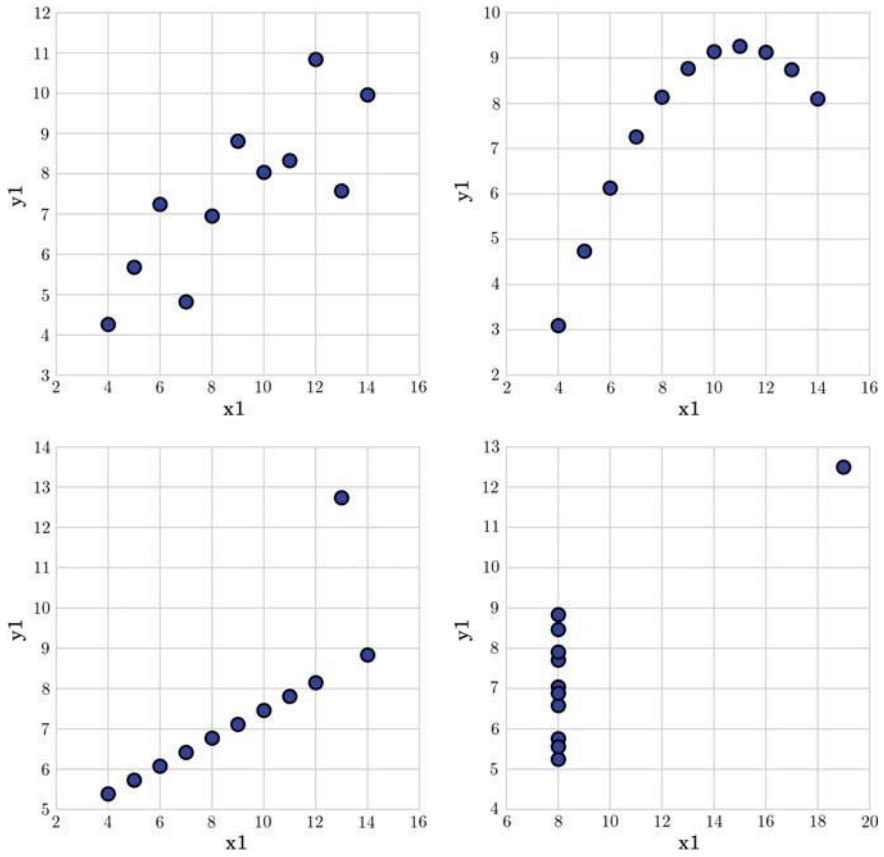


Fig. 3.10 Anscombe configurations

having ρ 0, does not necessarily mean that the variables are not correlated! Pearson's correlation captures correlations of first order, but not nonlinear correlations.Moreover, it does not work well in the presence of outliers.

Spearman's Rank Correlation

The *Spearman's rank correlation* comes as a solution to the robustness problem of Pearson's correlation when the data contain outliers. The main idea is to use the ranks of the sorted sample data, instead of the values themselves. For example, in the list [4, 3, 7, 5], the rank of 4 is 2, since it will appear second in the ordered list ([3, 4, 5, 7]). Spearman's correlation computes the correlation between the ranks

of the data. For example, considering the data: X = [10, 20, 30, 40, 1000], and Y = [70, 1000, 50, 10, 20], where we have an outlier in each one set. If we compute the ranks, they are [1.0, 2.0, 3.0, 4.0, 5.0] and [2.0, 1.0, 3.0, 5.0, 4.0]. As value of the Pearson's coefficient, we get 0.28, which does not show much

between the sets. However, the Spearman's rank coefficient, capturing the correlation between the ranks, gives as a final value of 0.80, confirming the correlation betweenthe sets. As an exercise, you can compute the Pearson's and the Spearman's rank correlations for the different Anscombe configurations given in Fig. 3.10. Observe if linear and nonlinear correlations can be captured by the Pearson's and the Spearman's rank correlations.

Statistical Inference

Introduction

There is not only one way to address the problem of statistical inference. In fact, there are two main approaches to statistical inference: the frequentist and Bayesianapproaches. Their differences are subtle but fundamental:

- In the case of the *frequentist approach*, the main assumption is that there is a population, which can be represented by several parameters, from which we can obtain numerous random samples. Population parameters are fixed but they are not accessible to the observer. The only way to derive information about these parameters is to take a sample of the population, to compute the parameters of thesample, and to use statistical inference techniques to make probable propositionsregarding population parameters.
- The *Bayesian approach* is based on a consideration that data are fixed, not the result of a repeatable sampling process, but parameters describing data can be described probabilistically. To this end, Bayesian inference methods focus on producing parameter distributions that represent all the knowledge we can extract from the sample and from prior information about the problem.

A deep understanding of the differences between these approaches is far beyondthe scope of this chapter, but there are many interesting references that will enable you to learn about it [1]. What is really important is to realize that the approaches are based on different assumptions which determine the validity of their inferences.The assumptions are related in the first case to a sampling process; and to a statistical model in the second case. Correct inference requires these assumptions to be correct. The fulfillment of this requirement is not part of the method, but it is the responsibility of the data scientist.

In this chapter, to keep things simple, we will only deal with the first approach, but we suggest the reader also explores the second approach as it is well worth it!

Statistical Inference: The Frequentist Approach

As we have said, the ultimate objective of statistical inference, if we adopt the fre- quentist approach, is to produce probable propositions concerning population

param- eters from analysis of a sample. The most important classes of propositions are as follows:

- Propositions about *point estimates*. A point estimate is a particular value that best approximates some parameter of interest. For example, the mean or the varianceof the sample.
- Propositions about *confidence intervals* or *set estimates*. A confidence interval isa range of values that best represents some parameter of interest.
- Propositions about the acceptance or rejection of a *hypothesis*.

In all these cases, the production of propositions is based on a simple assumption: we can estimate the probability that the result represented by the proposition has been caused by chance. The estimation of this probability by sound methods is oneof the main topics of statistics.

The development of traditional statistics was limited by the scarcity of computa- tional resources. In fact, the only computational resources were mechanical devices and human computers, teams of people devoted to undertaking long and tedious calculations. Given these conditions, the main results of classical statistics are theo- retical approximations, based on idealized models and assumptions, to measure the effect of chance on the statistic of interest. Thus, concepts such as the *Central Limit Theorem*, the *empirical sample distribution* or the *t-test* are central to understandingthis approach.

The development of modern computers has opened an alternative strategy for measuring chance that is based on simulation; producing computationally inten- sive methods including resampling methods (such as bootstrapping), Markov chain Monte Carlo methods, etc. The most interesting characteristic of these methods is that they allow us to treat more realistic models.

Measuring the Variability in Estimates

Estimates produced by descriptive statistics are not equal to the *truth* but they are better as more data become available. So, it makes sense to use them as central elements of our propositions and to measure its variability with respect to the sample size.

Point Estimates

Let us consider a dataset of accidents in Barcelona in 2013. This dataset can be downloaded from the OpenDataBCN website,[1] Barcelona City Hall's open data service. Each register in the dataset represents an accident via a series of features: weekday, hour, address, number of dead and injured people, etc. This dataset will represent our population: the set of all reported traffic accidents in Barcelona during2013.

In[1]:

Sampling Distribution of Point Estimates

Let us suppose that we are interested in describing the daily number of traffic acci- dents in the streets of Barcelona in 2013. If we have access to the *population*, the computation of this parameter is a simple operation: the total number of accidents divided by 365.

```
data = pd.read_csv("files/ch04/ACCIDENTS_GU_BCN_2013.csv")
data['Date'] = data[u'Dia de mes'].apply(lambda x: str(x))
               + '-' +
               data[u'Mes de any'].apply(lambda x: str(x))
data['Date'] = pd.to_datetime(data['Date'])
```

Out[1]:
Mean:
25.9095

suppose that we only have access to a limited part of the data (the *sample*): the number of accidents during *some* days of 2013. Can we still give an approximation of the population mean?

The most intuitive way to go about providing such a mean is simply to take the *sample mean*. The sample mean is a point estimate of the population mean. If we can only choose one value to estimate the population mean, then this is our best guess.

The problem we face is that estimates generally vary from one sample to another, and this sampling variation suggests our estimate may be close, but it will not be exactly equal to our parameter of interest. How can we measure this variability?

In our example, because we have access to the population, we can empirically build the *sampling distribution of the sample mean*[2] for a given number of observations. Then, we can use the sampling distribution to compute a measure of the variability. In Fig. 4.1, we can see the empirical sample distribution of the mean for *s* = 10.000 samples of 200 observations from our dataset. This empirical distribution has been built in the following way:

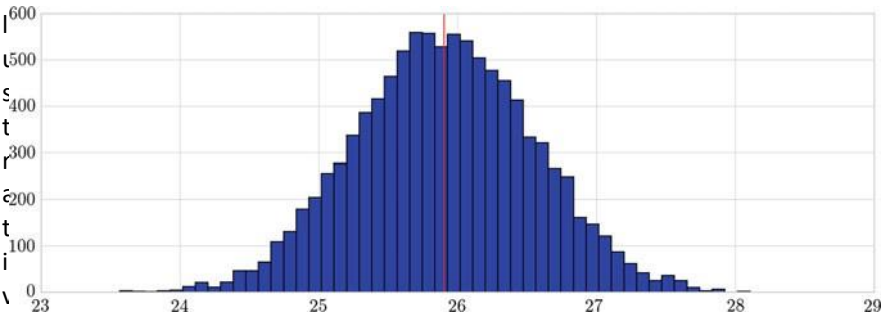


Fig. 4.1 Empirical distribution of the sample mean. In *red*, the mean value of this distribution

1. Draw *s* (a large number) independent samples $\{\mathbf{x}^1, \ldots, \mathbf{x}^s\}$ from the population where each element $\mathbf{x}^j$ is composed of $\{x_i^j\}_{i=1,\ldots,n}$.
2. Evaluate the sample mean $\hat{\mu}^j = \frac{1}{n}\sum_{i=1}^{n} x_i^j$ of each sample.
3. Estimate the sampling distribution of μ by the empirical distribution of the sample replications.

```
# population
df = accidents.to_frame()
N_test = 10000
elements = 200
# mean array of samples
means = [0] * N_test
# sample generation
for i in range(N_test):
    rows = np.random.choice(df.index.values, elements)
    sampled_df = df.ix[rows]
    means[i] = sampled_df.mean()
```

te from a sample of size n, we define its *sampling distribution* as the distribution of the point estimate based on samples of size n from its population. This definition is valid for point estimates of other population parameters, such as the population median or population standard deviation, but we will focus on the analysis of the sample mean.

In general, given a point estimate, we can define its sampling distribution as the distribution of the point estimate based on samples of size n from its population. This definition is valid for point estimates of other population parameters, such as the population median or population standard deviation, but we will focus on the analysis of the sample mean.

The sampling distribution of an estimate plays an important role in understanding the real meaning of propositions concerning point estimates. It is very useful to think of a particular point estimate as being drawn from such a distribution.

The Traditional Approach

In real problems, we do not have access to the real population and so estimation of the sampling distribution of the estimate from the empirical distribution of the sample replications is not an option. But this problem can be solved by making use of some theoretical results from traditional statistics.

It can be mathematically shown that given n independent observations $\{x_{i\ i=1,..,n}\}$ of a population with a standard deviation σ_x, the standard deviation of the samplemean $\sigma_{\bar{x}}$, or *standard error*, can be approximated by this formula:

$$SE = \frac{\sigma_x}{\sqrt{n}}$$

The demonstration of this result is based on the Central Limit Theorem: an oldtheorem with a history that starts in 1810 when Laplace released his first paper on it.This formula uses the standard deviation of the population σ_x, which is not known, but it can be shown that if it is substituted by its empirical estimate $\hat{\sigma}_x$, the estimationis sufficiently good if $n > 30$ and the population distribution is not skewed. Thisallows us to estimate the standard error of the sample mean even if we do not have access to the population.

So, how can we give a measure of the variability of the sample mean? The answeris simple: by giving the **empirical standard error of the mean distribution**.

```
rows = np.random.choice(df.index.values, 200)
sampled_df = df.ix[rows]
est_sigma_mean = sampled_df.std()/math.sqrt(200)

print 'Direct estimation of SE from one sample of
      200 elements:', est_sigma_mean[0]
print ' Estimation of the SE by simulating 10000 samples of
      200 elements:', np.array(means).std()
```

Out[3]: Direct estimation of SE from one sample of 200 elements: 0.6536Estimation of the SE by simulating 10000 samples of 200
elements: 0.6362

Unlike the case of the sample mean, there is no formula for the standard error ofother interesting sample estimates, such as the median.

The Computationally Intensive Approach

Let us consider from now that our full dataset is a sample from a hypothetical population (this is the most common situation when analyzing real data!).

A modern alternative to the traditional approach to statistical inference is the bootstrapping method [2]. In the bootstrap, we draw *n* observations *with replacement* from the original data to create a bootstrap sample or resample. Then, we can calculate the mean for this resample. By repeating this process a large number of times, we can build a good approximation of the mean sampling distribution (see Fig. 4.2).

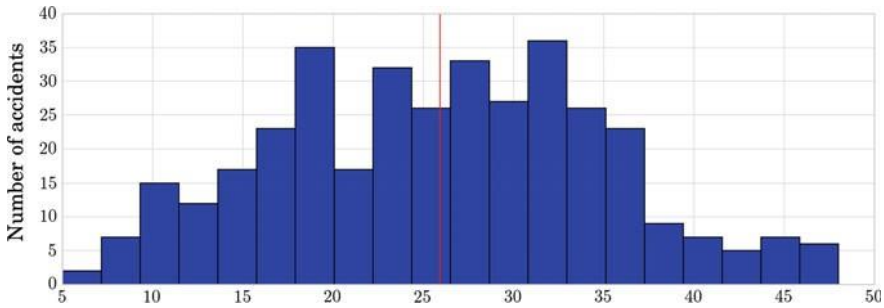


Fig. 4.2 Mean sampling distribution by bootstrapping. In *red*, the mean value of this distribution

In [4]:

```
def meanBootstrap(X, numberb):
    x = [0]*numberb
    for i in range(numberb):
        sample = [X[j]
                  for j
                  in np.random.randint(len(X), size=len(X))
                 ]
        x[i] = np.mean(sample)
    return x
m = meanBootstrap(accidents, 10000)
print "Mean estimate:", np.mean(m)
```

Out[4]: Mean estimate: 25.9094

The basic idea of the bootstrapping method is that the observed sample contains sufficient information about the underlying distribution. So, the information we can extract from resampling the sample is a good approximation of what can be expected from resampling the population.

The bootstrapping method can be applied to other simple estimates such as the median or the variance and also to more complex operations such as estimates of censored data.[3]

Confidence Intervals

A point estimate Θ, such as the sample mean, provides a *single plausible value for a parameter*. However, as we have seen, a point estimate is rarely perfect; usually there is some error in the estimate. That is why we have suggested using the standard error as a measure of its variability.

Instead of that, a next logical step would be to provide *a plausible range of values* for the parameter. A plausible range of values for the sample parameter is called a *confidence interval*.

We will base the definition of *confidence interval* on two ideas:

1. Our point estimate is the most plausible value of the parameter, so it makes senseto build the confidence interval around the point estimate.
2. The *plausibility* of a range of values can be defined from the sampling distributionof the estimate.

For the case of the mean, the Central Limit Theorem states that its samplingdistribution is normal:

Theorem 4.1 *Given a population with a finite mean μ and a finite non-zero variance σ² , the sampling distribution of the mean approaches a normal distribution with a mean of μ and a variance of σ²/n as n, the sample size, increases.*

In this case, and in order to define an interval, we can make use of a well-known result from probability that applies to normal distributions: roughly 95% of the time our estimate will be within 1.96 standard errors of the true mean of the distribution. If the interval spreads out 1.96 standard errors from a normally distributed point estimate, intuitively we can say that we are *roughly 95% confident that we have captured the true parameter*.

$$CI = [\Theta - 1.96 \times SE, \Theta + 1.96 \times SE]$$

In[5]:

```
m = accidents.mean()
se = accidents.std()/math.sqrt(len(accidents))
ci = [m - se*1.96, m + se*1.96]
print "Confidence interval:", ci
```

Out[5]: Confidence interval: [24.975, 26.8440]

Suppose we want to consider confidence intervals where the confidence level issomewhat higher than 95%: perhaps we would like a confidence level of 99%. To create a 99% confidence interval, change 1.96 in the 95% confidence interval formula to be 2.58 (it can be shown that 99% of the time a normal random variable will be within 2.58 standard deviations of the mean).

In general, if the point estimate follows the normal model with standard error SE , then a confidence interval for the population parameter is

$$\Theta \pm z \times SE$$

where z corresponds to the confidence level selected:

Confidence Level	90%	95%	99%	99.9%
z Value	1.65	1.96	2.58	3.291

This is how we would compute a 95% confidence interval of the sample mean using bootstrapping:

1. Repeat the following steps for a large number, *s*, of times:

    a. Draw *n* observations with replacement from the original data to create abootstrap sample or resample.
    b. Calculate the mean for the resample.

2. Calculate the mean of your *s* values of the sample statistic. This process givesyou a "bootstrapped" estimate of the sample statistic.
3. Calculate the standard deviation of your *s* values of the sample statistic. Thisprocess gives you a "bootstrapped" estimate of the SE of the sample statistic.
4. Obtain the 2.5th and 97.5th percentiles of your *s* values of the sample statistic.

In [6]:

```
m = meanBootstrap(accidents, 10000)
sample_mean = np.mean(m)
sample_se =  np.std(m)

print "Mean estimate:", sample_mean
print "SE of the estimate:",  sample_se

ci = [np.percentile(m, 2.5), np.percentile(m, 97.5)]
print  "Confidence interval:", ci
```

Out[6]: Mean estimate: 25.9039
SE of the estimate: 0.4705
Confidence interval: [24.9834, 26.8219]

But What Does "95% Confident" Mean?

The real meaning of "confidence" is not evident and it must be understood from thepoint of view of the generating process.

Suppose we took many (infinite) samples from a population and built a 95% confidence interval from each sample. Then about 95% of those intervals would contain the actual parameter. In Fig. 4.3 we show how many confidence intervals computed from 100 different samples of 100 elements from our dataset contain the real population mean. If this simulation could be done with infinite different samples, 5% of those intervals would not contain the true mean.

So, when faced with a sample, the correct interpretation of a confidence intervalis as follows:

> In 95% of the cases, when I compute the 95% confidence interval from this sample, the true mean of the population will fall within the interval defined by these bounds: ±1.96 × *SE*.

We cannot say either that our specific sample contains the true parameter or that the interval has a 95% chance of containing the true parameter. That interpretation would not be correct under the assumptions of traditional

statistics.

Hypothesis Testing

Giving a measure of the variability of our estimates is one way of producing a statistical proposition about the population, but not the only one. R.A. Fisher (1890–1962) proposed an alternative, known as *hypothesis testing*, that is based on the concept of *statistical significance*.

Let us suppose that a deeper analysis of traffic accidents in Barcelona results in a difference between 2010 and 2013. Of course, the difference could be caused only by chance, because of the variability of both estimates. But it could also be the case that traffic conditions were very different in Barcelona during the two periods and, because of that, data from the two periods can be considered as belonging to two different populations. Then, the relevant question is: Are the observed effects real ornot?

Technically, the question is usually translated to: *Were the observed effects statistically significant*?

The process of determining the statistical significance of an effect is called *hypothesis testing*.

This process starts by simplifying the options into two competing hypotheses:

- H_0: The mean number of daily traffic accidents is the same in 2010 and 2013 (there is only one population, one true mean, and 2010 and 2013 are just differentsamples from the same population).
- H_A: The mean number of daily traffic accidents in 2010 and 2013 is different (2010 and 2013 are two samples from two different populations).

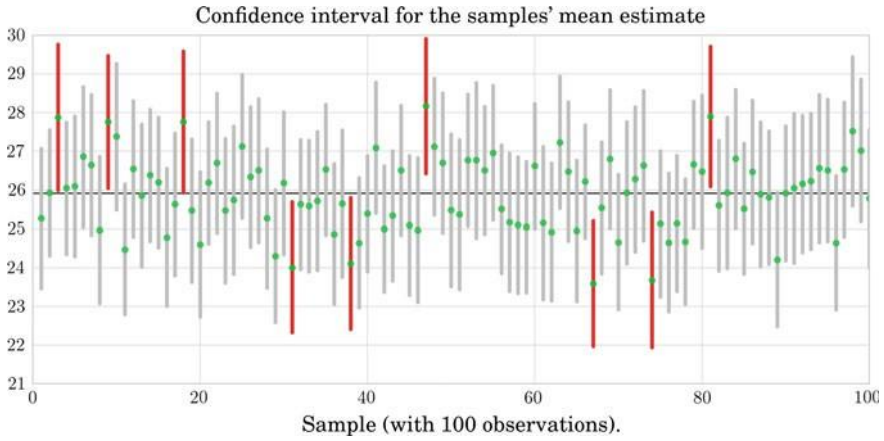


Fig. 4.3 This graph shows 100 sample means (*green points*) and its corresponding confidence intervals, computed from 100 different samples of 100 elements from our dataset. It can be observed that a few of them (those in *red*) do not contain the mean of the population (*black horizontal line*)

We call H_0 the *null hypothesis* and it represents a *skeptical* point of view: the effect we have observed is due to chance (due to the specific sample bias). H_A is the*alternative hypothesis* and it represents the other point of view: the effect is real.

The general rule of frequentist hypothesis testing: we will not *discard* H_0 (and hence we will not consider H_A) unless the observed effect is *implausible* under H_0.

Testing Hypotheses Using Confidence Intervals

We can use the concept represented by *confidence intervals* to measure the *plausi-bility* of a hypothesis.

We can illustrate the evaluation of the hypothesis setup by comparing the meanrate of traffic accidents in Barcelona during 2010 and 2013:

In[7]:
```
data = pd.read_csv("files/ch04/ACCIDENTS_GU_BCN_2010.csv",
                   encoding='latin-1')

# Create a new column which is the date
data['Date'] = data['Dia de mes'].apply(lambda x: str(x))
               + '-' +
               data['Mes de any'].apply(lambda x: str(x))
data2 = data['Date']
counts2010 = data['Date'].value_counts()
print '2010: Mean', counts2010.mean()

data = pd.read_csv("files/ch04/ACCIDENTS_GU_BCN_2013.csv",
                   encoding='latin-1')

# Create a new column which is the date
data['Date'] = data['Dia de mes'].apply(lambda x: str(x))
               + '-' +
               data['Mes de any'].apply(lambda x: str(x))
data2 = data['Date']
counts2013 = data['Date'].value_counts()
print '2013: Mean', counts2013.mean()
```

Out[7]:
```
2010: Mean 24.8109
2013: Mean 25.9095
```

This estimate suggests that in 2013 the mean rate of traffic accidents in Barcelonawas higher than it was in 2010. But is this effect statistically significant?

Based on our sample, the 95% confidence interval for the mean rate of trafficaccidents in Barcelona during 2013 can be calculated as follows:

In[8]:
```
n = len(counts2013)
mean = counts2013.mean()
s = counts2013.std()
ci = [mean - s*1.96/np.sqrt(n), mean + s*1.96/np.sqrt(n)]
print '2010 accident rate estimate:', counts2010.mean()
print '2013 accident rate estimate:', counts2013.mean()
print 'CI for 2013:',ci
```

```
Out[8]: 2010 accident rate estimate: 24.8109
        2013 accident rate estimate: 25.9095
CI for 2013: [24.9751, 26.8440]
```

Because the 2010 accident rate estimate does not fall in the range of plausible values of 2013, we say the alternative hypothesis cannot be discarded. That is, it cannot be ruled out that in 2013 the mean rate of traffic accidents in Barcelona washigher than in 2010.

Interpreting CI Tests

Hypothesis testing is built around rejecting or failing to reject the null hypothesis. That is, we do not reject H_0 unless we have strong evidence against it. But what precisely does strong evidence mean? As a general rule of thumb, for those cases where the null hypothesis is actually true, we do not want to incorrectly reject H_0 more than 5% of the time. This corresponds to a *significance level* of $\alpha = 0.05$. In this case, the correct interpretation of our test is as follows:

> If we use a 95% confidence interval to test a problem where the null hypothesis is true, we will make an error whenever the point estimate is at least 1.96 standard errors away from thepopulation parameter. This happens about 5% of the time (2.5% in each tail).

Testing Hypotheses Using *p*-Values

A more advanced notion of *statistical significance* was developed by R.A. Fisher in the 1920s when he was looking for a test to decide whether variation in crop yields was due to some specific intervention or merely random factors beyond experimental control.

Fisher first assumed that fertilizer caused no difference (*null hypothesis*) and thencalculated *P*, the probability that an observed yield in a fertilized field would occurif fertilizer had no real effect. This probability is called the *p-value*.

The *p*-value is the probability of observing data at least as favorable to the alter-native hypothesis as our current dataset, if the null hypothesis is true. We typically use a summary statistic of the data to help compute the *p*-value and evaluate the hypotheses.

Usually, if *P* is less than 0.05 (the chance of a fluke is less than 5%) the result is declared *statistically significant*.

It must be pointed out that this choice is rather arbitrary and should not be takenas a scientific truth.

The goal of classical hypothesis testing is to answer the question, "*Given a sample and an apparent effect, what is the probability of seeing such an effect by chance?*"Here is how we answer that question:

- The first step is to quantify the size of the apparent effect by choosing a test statistic. In our case, the apparent effect is a difference in accident rates, so a natural choicefor the test statistic is the **difference in means between the two periods**.

- The second step is to define a *null hypothesis*, which is a model of the system based on the assumption that the apparent effect is not real. In our case, the null hypothesis is that there is no difference between the two periods.
- The third step is to compute a *p-value*, which is the probability of seeing the apparent effect if the null hypothesis is true. In our case, we would compute the difference in means, then compute the probability of seeing a difference as big, orbigger, under the null hypothesis.
- The last step is to *interpret the result*. If the *p*-value is low, the effect is said to be *statistically significant*, which means that it is unlikely to have occurred by chance. In this case we infer that the effect is more likely to appear in the larger population.

In our case, the test statistic can be easily computed:

In [9]:
```
m= len(counts2010)
n= len(counts2013)
p = (counts2013.mean() - counts2010.mean())
print 'm:', m, 'n:', n
print 'mean difference: ', p
```

Out[9]: m: 365 n: 365
mean difference: 1.0986

To approximate the *p*-value , we can follow the following procedure:

1. Pool the distributions, generate samples with size n and compute the differencein the mean.
2. Generate samples with size n and compute the difference in the mean.
3. Count how many differences are larger than the observed one.

In [10]:
```
# pooling distributions
x = counts2010
y = counts2013
pool = np.concatenate([x, y])
np.random.shuffle(pool)

#sample generation
import random
N = 10000 # number of samples
diff = range(N)
for i in range(N):
    p1 = [random.choice(pool) for _ in xrange(n)]
    p2 = [random.choice(pool) for _ in xrange(n)]
    diff[i] = (np.mean(p1) - np.mean(p2))
```

In [11]:

```
# counting differences larger than the observed one
diff2 = np.array(diff)
w1 = np.where(diff2 > p)[0]

print 'p-value (Simulation)=', len(w1)/float(N),
      '(', len(w1)/float(N)*100 ,'%)', 'Difference =', p
if (len(w1)/float(N)) < 0.05:
    print 'The effect is likely'
else:
    print 'The effect is not likely'
```

Out[11]: p-value (Simulation)= 0.0485 (4.85%) Difference = 1.098The effect is likely

Interpreting *P*-Values

A *p*-value is the probability of an observed (or more extreme) result arising only from chance.

If *P* is less than 0.05, there are two possible conclusions: there is a real effect or the result is an improbable fluke. *Fisher's method offers no way of knowing which isthe case*.

We must not confuse the odds of getting a result (if a hypothesis is true) with the odds of favoring the hypothesis if you observe that result. If *P* is less than 0.05,we cannot say that this means that it is 95% certain that the observed effect is real and could not have arisen by chance. Given an observation E and a hypothesis H, $P(E|H)$ and $P(H|E)$ are not the same!

Another common error equates *statistical significance* to *practical importance/ relevance*. When working with large datasets, we can detect statistical significancefor small effects that are meaningless in practical terms.

We have defined the effect as *a difference in mean as large or larger than δ, considering the sign*. A test like this is called *one sided*.

If the relevant question is whether *accident rates are different*, then it makes senseto test the absolute difference in means. This kind of test is called two sided becauseit counts both sides of the distribution of differences.

Direct Approach

The formula for the standard error of the absolute difference in two means is similarto the formula for other standard errors. Recall that the standard error of a single mean can be approximated by:

$$SE_{\bar{x}_1} = \frac{\sigma_1}{\sqrt{n_1}}$$

The standard error of the difference of two sample means can be constructed fromthe standard errors of the separate sample means:

$$SE_{\bar{x}_1 - \bar{x}_2} = \sqrt{\frac{\sigma_1^2}{n_1} + \frac{\sigma_2^2}{n_2}}$$

This would allow us to define a direct test with the 95% confidence interval.

But Is the Effect *E* Real?

We do not yet have an answer for this question! We have defined a null hypothesis H_0 (the effect is not real) and we have computed the probability of the observed effect under the null hypothesis, which is $P(E \mid H_0)$, where E is an effect as big as or bigger than the apparent effect and a p-value .

We have stated that from the frequentist point of view, we cannot consider H_A unless $P(E \mid H_0)$ is less than an arbitrary value. But the real answer to this question must be based on comparing $P(H_0 \mid E)$ to $P(H_A \mid E)$, not on $P(E \mid H_0)$! One possi- ble solution to these problems is to use *Bayesian reasoning*; an alternative to the frequentist approach.

No matter how many data you have, you will still depend on intuition to decide how to interpret, explain, and use that data. Data cannot speak by themselves. Data scientists are interpreters, offering one interpretation of what the useful narrative story derived from the data is, if there is one at all.